

Gradient Descent'in Matematiksel Arka Planı

Kodlamadan önce sindirilecek temel çalışma kağıdı — Miuul Makine Öğrenmesi Modülü

Bu çalışma kağıdının amacı: Gradient descent algoritmasının koduna geçmeden önce, "neden bu formül böyle" sorusunun cevabını kağıt üzerinde, elle hesaplayarak kalıcı biçimde oturtmak. Her formülün önce sezgisi, sonra matematiği verilecek. Alıştırmaları atlamadan, sırayla, kalemle çöz.

1 Problem Nedir?

Elimizde bağımsız değişken x (örneğin TV reklam harcaması) ile bağımlı değişken y (satış) arasında bir doğrusal ilişki kurmak istiyoruz. Aradığımız doğru şu genel formda:

HİPOTEZ FONKSİYONU (MODELİN TAHMİNİ)

$$h_{\theta}(x) = \theta_0 + \theta_1 \cdot x$$

- θ_0 (theta-sıfır): doğrunun y-eksenini kestiği nokta (sabit terim / bias)
- θ_1 (theta-bir): doğrunun eğimi (x bir birim arttığında y ne kadar değişiyor)
- $h_{\theta}(x)$: bu θ değerleriyle modelin ürettiği tahmin

Sorun şu: **sonsuz sayıda (θ_0, θ_1) çifti**, dolayısıyla sonsuz sayıda doğru mümkün. Hangisinin "en iyi" olduğuna nasıl karar vereceğiz? Önce "iyilik"i ölçmemiz, sonra da bu ölçüyü en iyi yapan θ değerlerini **bulmamız** gerekiyor. Gradient descent, ikinci kısmı yani "bulma" işini yapan yöntemdir.

2 Maliyet Fonksiyonu $J(\theta_0, \theta_1)$

Önce modelin ne kadar "kötü" olduğunu tek bir sayıyla ifade eden bir fonksiyon tanımlayalım:

MALİYET (COST) FONKSİYONU

$$J(\theta_0, \theta_1) = (1/2m) \cdot \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Parça parça okuma

- $i = 1 \dots m$: Veri setindeki her satır (gözlem). m = toplam gözlem sayısı.
- $h_{\theta}(x^{(i)}) - y^{(i)}$: O gözlem için tahmin ile gerçek değer arasındaki fark (hata).
- $(\dots)^2$: Bu farkın karesi.
- Σ : Tüm gözlemlerdeki kare hataları topla.

5 **1/(2m):** Toplamı ortalamaya çevir; 2'ye bölme, sadece ileride türev alırken sadeleşme sağlayan bir kolaylık.

NEDEN?

Hatayı doğrudan toplamak yerine **kareye** alıyoruz çünkü: (a) negatif ve pozitif hatalar birbirini götürüp yanlış bir "sıfır hata" izlenimi vermesin, (b) büyük hatalar kareleyince orantısız büyür, yani model büyük sapmalardan çok daha fazla "rahatsız" olsun.

Sayısal örnek

Veri: (1,3), (2,5), (3,7). $\theta_0=0$, $\theta_1=1$ seçelim (yani doğru: $y = x$).

x	y (gerçek)	$h(x) = \theta_0 + \theta_1 x$	$\text{hata} = h(x) - y$	hata^2
1	3	1	-2	4
2	5	2	-3	9
3	7	3	-4	16

Toplam kare hata = $4+9+16 = 29$. $m=3$ olduğuna göre: $J = 29 / (2 \cdot 3) = 4.83$.

ALIŞTIRMA 1

Aynı veri (1,3), (2,5), (3,7) için $\theta_0=1$, $\theta_1=2$ seçersen J kaç çıkar? Tabloyu kendin kur, hesapla.

✓ CEVAP ANAHTARI

Tahminler: $h(1)=1+2=3$, $h(2)=1+4=5$, $h(3)=1+6=7$ — hepsi gerçek değerle birebir aynı. Hatalar: 0,0,0. $J = 0/(2 \cdot 3) = 0$. Bu, bu veri için mümkün olan en iyi (mükemmel) sonuçtur.

3 Amaç: J'yi Minimize Etmek

OPTİMİZASYON HEDEFİ

minimize $J(\theta_0, \theta_1)$ θ_0, θ_1 üzerinden

J, veriye değil **θ_0 ve θ_1 'in seçimine bağlı bir fonksiyondur**. Veri sabit kalır; sen θ değerlerini değiştirdikçe J'nin çıktısı değişir. Amacımız, J'yi mümkün olan en küçük değere indiren θ_0, θ_1 çiftini bulmak.

SEZGİ / BENZETME

Bunu bir **çukur/vadi** gibi düşün. Yatay eksen: θ 'nin alabileceği değerler. Dikey eksen: J değeri. θ seçildiğinde J'nin ne kadar çıktığı. θ , gerçek/optimal değerden uzaklaştıkça J büyür (çukurun kenarlarına tırmanırsın); θ optime yaklaştıkça J küçülür (çukurun dibine inersin). Bu şekle **konveks (tek dipli, kase şeklinde) fonksiyon** denir — kare aldığımız için grafiği hep böyle çıkar, birden fazla çukuru olmaz. Bu önemli: tek bir dip olduğu için "yanlış çukura düşme" riski yok.

4 Dibe Nasıl İnilir? — Gradyan (Türev) Sezgisi

SEZGİ / BENZETME

Sisli bir dağdasın, aşağıyı göremiyorsun, sadece **ayağının altındaki eğimi hissedebiliyorsun**. Vadiye inmek istiyorsun. Mantıklı strateji: ayağının altı hangi yöne "aşağı" eğimliyse o yöne bir adım at. Ayağının altındaki bu eğim ölçüsüne matematikte **türev (gradyan)** denir.

Eğimin yönü ne anlatır?

Eğim durumu	Anlamı	Ne yapmalı?
Eğim pozitif	Sağa gittikçe fonksiyon yükseliyor	Aşağı inmek için SOLA git (θ 'yi küçült)
Eğim negatif	Sağa gittikçe fonksiyon alçalıyor	Aşağı inmek için SAĞA git (θ 'yi büyüt)

NEDEN?

Güncelleme kuralı her iki durumu da **tek bir formülle** otomatik halleder: **yeni θ = eski θ – eğim**. Eğim pozitifse çıkarınca θ küçülür (sola gider); eğim negatifse çıkarınca θ büyür (sağa gider, çünkü eksi bir sayıyı çıkarmak toplamaya eşittir).

5 Kısmi Türevlerin Türetilmesi

J'nin θ_0 'a göre ve θ_1 'e göre kısmi türevini alacağız. Bu, " θ_0 'ı biraz kıpırdatarsam J ne kadar değişir" ve " θ_1 'i biraz kıpırdatarsam J ne kadar değişir" sorularının cevabıdır.

θ_0 'a göre türev

$J = (1/2m) \cdot \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2$ ifadesinde $h_{\theta}(x) = \theta_0 + \theta_1 x$ olduğunu yerine koyarsak, iç fonksiyonun θ_0 'a göre türevi 1'dir (çünkü θ_0 'ın katsayısı 1). Zincir kuralı uygulanınca:

$$\partial J / \partial \theta_0 = (1/m) \cdot \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

(Kare alma işleminin türevi 2 kat verir, ama önden koyduğumuz $1/2$ bu 2 ile sadeleşir — **2m yerine m kalmasının sebebi tam olarak budur.**)

θ_1 'e göre türev

Bu sefer iç fonksiyonun θ_1 'e göre türevi x 'tir (çünkü θ_1 'in katsayısı x). Zincir kuralı gereği sona bir x çarpanı daha eklenir:

$$\partial J / \partial \theta_1 = (1/m) \cdot \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

NEDEN?

θ_1 formülünde neden sonda x var, θ_0 'da yok? θ_0 sabit bir terim (x ile çarpılı değil), θ_1 ise x ile çarpılı bir terim. Türev alırken bir değişkenin "yanındaki çarpan" türevin sonucuna otomatik olarak dahil olur. Sezgisel olarak da: x büyükse (örneğin TV harcaması çok yüksekse), o gözlemdeki hatanın θ_1 'i değiştirmedeki payı da büyük olsun istiyoruz.

6 Güncelleme Kuralları (Asıl Adım)

SİMÜLTANE GÜNCELLEME

$$\theta_0 := \theta_0 - \alpha \cdot (1/m) \cdot \sum (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \cdot (1/m) \cdot \sum (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

" $:=$ " **işareti** matematiksel eşitlik değil, programlamadaki **atama** anlamına gelir: "sağdaki değeri hesapla, soldakinin yeni değeri yap".

Kritik kural — Eşzamanlı (simultaneous) güncelleme: θ_0 ve θ_1 , her ikisi de **eski değerleriyle** hesaplanmalı. Yani önce θ_0 'ı güncelleyip, θ_1 'i hesaplarken güncellenmiş θ_0 'ı kullanmak **yanlıştır**. Doğrusu: ikisinin de yeni değerini eski değerlerle hesapla, sonra ikisini birden güncelle.

α (alpha) — Öğrenme Oranı

SEZGİ / BENZETME

Dağcı benzetmesine dön: **α senin adım büyüklüğün.**

α seçimi	Sonuç
Çok küçük (örn. 0.0001)	Güvenli ama çok yavaş — vadiye inmek çok uzun sürer
Uygun (örn. 0.01 – 0.1)	Makul hızda, kontrollü şekilde dibe iner
Çok büyük (örn. 5)	Vadiyi ıskalar, karşı yamaca sıçrar; hatta J küçülmek yerine büyüyebilir (ıraksama)

Pratikte α , deneme yanılmayla seçilir: 0.001, 0.01, 0.1, 1 gibi değerler denenip J 'nin iterasyonlar boyunca düzgünce azaldığı değer seçilir.

7 Elle Çözülmüş Tam Örnek

Veri: (1,3), (2,5), (3,7) — $m=3$. Gerçek/optimal çözüm $\theta_0=1$, $\theta_1=2$ olduğunu biliyoruz (kontrol amaçlı).
Başlangıç: $\theta_0=0$, $\theta_1=0$, $\alpha=0.1$

İterasyon 1

x	y	$h(x)=\theta_0+\theta_1x$	$\text{hata}=h(x)-y$	$\text{hata} \cdot x$
1	3	0	-3	-3
2	5	0	-5	-10
3	7	0	-7	-21

$\text{Eğim}(\theta_0) = (-3-5-7)/3 = -5$ | $\text{Eğim}(\theta_1) = (-3-10-21)/3 = -34/3 \approx -11.33$

Yeni $\theta_0 = 0 - 0.1 \cdot (-5) = \mathbf{0.5}$ | Yeni $\theta_1 = 0 - 0.1 \cdot (-11.33) = \mathbf{1.133}$

ALİŞTİRMA 2

İterasyon 2'yi sen hesapla. $\theta_0=0.5$, $\theta_1=1.133$ ile başla, aynı tabloyu (x, y, h(x), hata, hata·x) kur, eğimleri bul, yeni θ_0 ve θ_1 'i hesapla. $\alpha=0.1$ kalsın.

✓ CEVAP ANAHTARI

$h(1)=0.5+1.133=1.633$, $\text{hata}=1.633-3=-1.367$

$h(2)=0.5+2.266=2.766$, $\text{hata}=2.766-5=-2.234$

$h(3)=0.5+3.399=3.899$, $\text{hata}=3.899-7=-3.101$

$\text{Eğim}(\theta_0) = (-1.367-2.234-3.101)/3 \approx -2.234$

$\text{Eğim}(\theta_1) = (-1.367 \cdot 1 - 2.234 \cdot 2 - 3.101 \cdot 3)/3 = (-1.367-4.468-9.303)/3 \approx -5.046$

Yeni $\theta_0 = 0.5 - 0.1 \cdot (-2.234) \approx \mathbf{0.723}$

Yeni $\theta_1 = 1.133 - 0.1 \cdot (-5.046) \approx \mathbf{1.638}$

Görüldüğü gibi θ_0 (1'e) ve θ_1 (2'ye) gitgide yaklaşıyor.

ALİŞTİRMA 3

İterasyon 3'ü de sen dene. $\theta_0=0.723$, $\theta_1=1.638$ ile devam et. Sonucun θ_0 'ın 1'e, θ_1 'in 2'ye biraz daha yaklaştığını göreceksin. Kaç iterasyonda "yeterince yakın" (örn. $J < 0.01$) olacağını merak ediyorsan, bunu bir sonraki adımda kodla test edeceksin — elle onlarca iterasyon yapmaya gerek yok, mantığı 2-3 iterasyonda görmek yeterli.

8 Bu Matematiğin Koda Bağlanması

Matematikteki adım	Kodda karşılığı
θ_0 , θ_1 başlangıç değeri (o gibi)	b, w değişkenlerini o (veya küçük rastgele) ile başlatmak

$h_{\theta}(x^{(i)})$ hesaplama	tahmin = $b + w * x$, tüm gözlemler için (vektörel işlem)
Hata: $h(x) - y$	tahmin - gerçek değer farkı
$\Sigma(\dots) \cdot (1/m)$	numpy'da <code>.mean()</code> veya <code>.sum()/m</code>
$\theta := \theta - \alpha \cdot \text{eğim}$	$b = b - \alpha * \text{eğim}_b$; $w = w - \alpha * \text{eğim}_w$
Bunu tekrarla (iterasyon)	for döngüsü, belirli sayıda tekrar (<code>num_iters</code>)
Dibe ulaşma kontrolü	J 'nin (cost) her iterasyonda ne kadar azaldığını yazdırıp izlemek

Önemli hatırlatma: `LinearRegression().fit()` bu iteratif süreci hiç kullanmaz — OLS (En Küçük Kareler) denen kapalı-form matematiksel formülle çukurun dibini tek hamlede hesaplar. Şimdi yazacağın gradient descent kodu, **aynı dibi farklı bir yoldan**, adım adım bularak sklearn'in bulduğu sonuca (yaklaşık) eşit bir b , w üretecek.

9 Kodlamaya Geçmeden Önce Kontrol Listesi

- ☐ $J(\theta_0, \theta_1)$ fonksiyonunun ne ölçtüğünü kendi cümlelerimle açıklayabiliyorum.
- ☐ Neden hata karesi alındığını (negatifleri yok etmek + büyük hataları cezalandırmak) biliyorum.
- ☐ Çukur/vadi benzetmesini ve "tek dipli (konveks)" olmasının anlamını anlıyorum.
- ☐ Eğimin (türevin) neden "yön" bilgisi verdiğini, ve neden ters yönde hareket ettiğimizi anlıyorum.
- ☐ θ_0 ve θ_1 formülleri arasındaki tek farkın (sondaki x çarpanı) nereden geldiğini biliyorum.
- ☐ α 'nın çok küçük/çok büyük olmasının ne gibi sorunlara yol açtığını açıklayabiliyorum.
- ☐ Elle en az 2 iterasyonu, hatasız şekilde, tablo kurarak hesapladım.
- ☐ Eşzamanlı güncelleme kuralının (θ_0 ve θ_1 'in ikisinin de eski değerle hesaplanması) ne demek olduğunu biliyorum.

Hepsini işaretleyebiliyorsan, kodlama kısmına geçmeye hazırsın. Takıldığın alıştırmaya olursa (özellikle 2. veya 3. iterasyon), çözümünü getir birlikte kontrol edelim.